ESADE × BANC SABADELL

CAPSTONE PROJECT 2026

Classical Machine Learning versus Large Language Models for Credit Scoring: Offline Simulation and Explainability

Final Capstone Report

Prepared by

Alessandro Mezzanotte · Jad Zoghaib · Francesc Cañavate Quero

Project Advisor

Josep Puig Sallés

July 2026

Executive Summary

Banc Sabadell asked whether a large language model can score consumer credit as well as the classical machine-learning models that have been the industry standard for over a decade. This capstone answers that question offline, as a research prototype with no production use, by putting a tuned gradient-boosting model (XGBoost) and OpenAI's GPT-5.4 through the same task on the same LendingClub loans (2012 to 2014), then comparing them on accuracy, default-catching (Charged-Off F1), ranking quality (AUC), realized profit and loss, stability, cost, and explainability.

The language-model side was explored through a six-lever protocol, choosing the model, the prompt, the cost setup, an ML-plus-LLM blend, and a retrieval-augmented memory, each an isolated experiment, before a single clean run against a sealed 1,000-loan test set that no tuning ever touched. Every stage was leakage-checked, and the final results carry significance tests. The central finding is a split decision: the two tools genuinely disagree on which loans to approve, so neither wins outright.

- **GPT-5.4 wins on accuracy and on money.** On the sealed test it scored 76 to 78% accuracy against XGBoost's 69%, and turned a realized profit of +$999k (plain prompt) to +$1.19M (chain-of-thought) where XGBoost lost roughly −$94k. Both the accuracy and the profit gaps are statistically significant.
- **XGBoost wins on default-catching and calibration.** It keeps the better Charged-Off F1 (0.381 vs 0.298) and the only usable risk-ranking probability (AUC 0.706). The LLM sounds 99.6% sure when it is right and 96.5% when it is wrong, so its confidence cannot support a tunable cut-off.
- **The advanced levers added little.** Neither blending the two models nor retrieval memory (RAG) clearly beat the simple setup, confirming that the structured features, not the modelling, are the ceiling on this data. Retrieval is therefore repurposed as an explainability aid: precedent loans are attached to a decision for a human to audit, not fed to the model to lift the score.
- **The simplest setup is the winner.** No engineered prompt reliably beat the plain one on re-runs, and prompt caching cut cost by about 90% with no accuracy loss; batching was rejected for collapsing accuracy and creating a fairness problem.

Bottom line. The LLM is the better fit exactly where this project sits, small samples and a modest ML model, and it delivers a plain-English reason with every decision, a natural fit for the GDPR right to an explanation. Classical ML remains preferable with large, clean, well-labelled data and wherever a calibrated, tunable risk score is required. The real deliverable is the method: a rigorous, leakage-safe comparison protocol any future model can be run through, kept as decision-support with a human in the loop, never an autopilot.

Table of Contents

1. Introduction

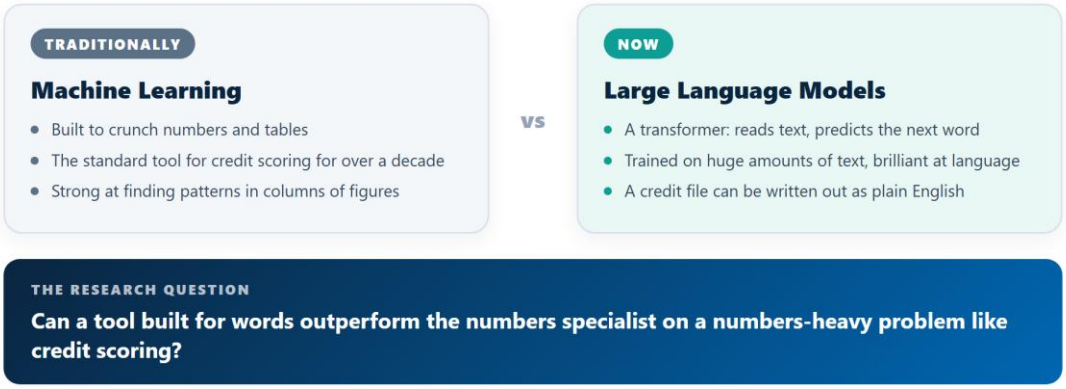
1.1 Project Context

This report documents the ESADE capstone project by Alessandro Mezzanotte, Jad Zoghaib, and Francesc Cañavate Quero, supervised by Josep Puig Sallés, in partnership with Banc Sabadell (IT & Operations).

Throughout the project, we compared a traditional machine learning model (gradient boosting) against a large language model's reasoned decision, produced through prompting, on anonymised credit files. Four dimensions were required, AUC, F1, stability, and cost, plus a side-by-side of explainability approaches: SHAP for the ML model against natural-language rationales for the LLM.

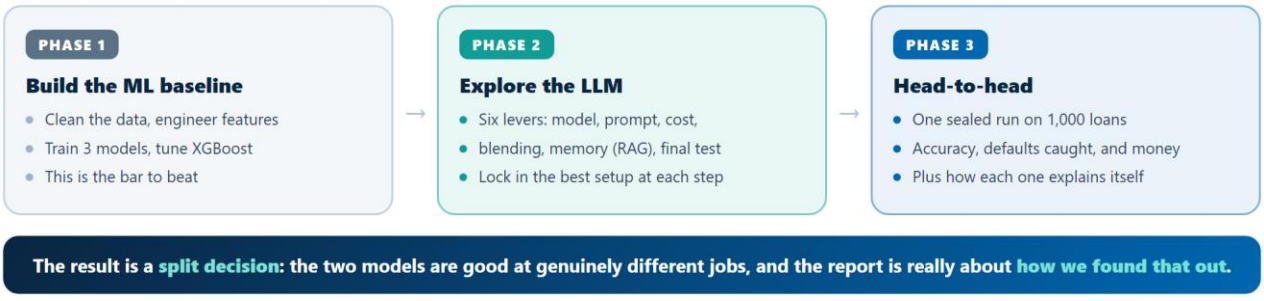
1.2 The Research Question

Credit scoring looks like a numbers problem: from a page of figures about a borrower, decide who repays and who defaults. For over a decade the standard tool has been tabular machine learning. Large language models are a different kind of tool, transformers trained to predict the next word, built for language rather than tables. But a credit file can be written out as language, which is what makes the comparison possible.



1.3 How the Project Fits Together

Answering the question meant building two comparable pipelines and putting them through the same test. The whole project runs in three phases: build a classical ML baseline (the bar to beat), then explore the LLM across six levers, then run the two head-to-head on a sealed set of loans nobody had touched. This report follows that same order.



1.4 Why Accuracy Alone Is Not Enough

Charged Off (default) loans are the minority class, roughly 15 to 17 percent of the dataset. A model that approved everyone would score 83 to 84 percent accuracy while catching zero defaults, so raw accuracy is misleading on its own. Three additional measures correct for this:

- Charged-Off precision, recall and F1 (how well the model finds the minority class)
- AUC (whether its probability ranks risky loans above safe ones)
- Realized profit and loss (what the lender actually earned or lost per loan, from LendingClub's recorded cashflows)

The financial lens matters most: as later sections show, the model that wins on F1 and AUC is not the model that wins on realized dollars, and reconciling that is one of the report's central findings.

2. Data and the Classical ML Baseline

2.1 The LendingClub Dataset

Both pipelines run on LendingClub's public accepted-loan dataset, restricted to loans issued between 2012 and 2014 that ended as either Fully Paid or Charged Off. The data is already de-identified. Each loan carries about thirty facts across three groups, all known at decision time so nothing from after the loan leaks in: borrower profile (income, employment, home ownership, FICO, debt-to-income), loan terms (amount, rate, term, instalment, grade, purpose), and credit history (open accounts, delinquencies, inquiries, public records).

Why that particular window? We picked 2012 to 2014 on purpose, because it is a calm, stable stretch of the economy, safely after the 2008 crisis and well before COVID. That choice is about fairness between the two models. The LLM arrives already knowing a great deal about the world, including recessions and credit cycles, while the ML model only knows the numbers in front of it. If we had included a crisis period, loan outcomes would be driven partly by a macro shock the ML model cannot see but the LLM might. Sticking to a stable window keeps the outcomes tied to the borrower and the loan, which both models see equally, so the comparison stays like-for-like.

Both models see the same features, so any gap in performance reflects the modelling approach rather than one side simply having more to work with.

2.2 Preprocessing

Before any model sees the data, we clean it up (02_Preprocessing.ipynb). The idea is simple: turn messy raw fields into numbers a model can use, without throwing away signal. The two tables below show the main moves, what a field looked like before, what we did, and why.

Field	Before	Treatment	Why
Interest rate	text like "13.99%"	plain number 13.99	so the model can do maths with it
Home ownership	rare ANY / NONE categories	merged into OTHER	too few of them to learn from alone
Grade and sub-grade	both present	kept sub-grade, dropped grade	sub-grade already contains the grade
Employment length	"< 1 year" … "10+ years"	a 0-to-10 scale	keeps the natural ordering intact
Public records, bankruptcies	raw counts	kept as counts, not yes/no flags	"3 bankruptcies" is worse than "1"
Issue date, credit line date	raw dates	one number: credit history in years	a clean figure instead of dates

Table 2. Cleaning and encoding. The frame ends up at 384,378 loans and 74 model features.

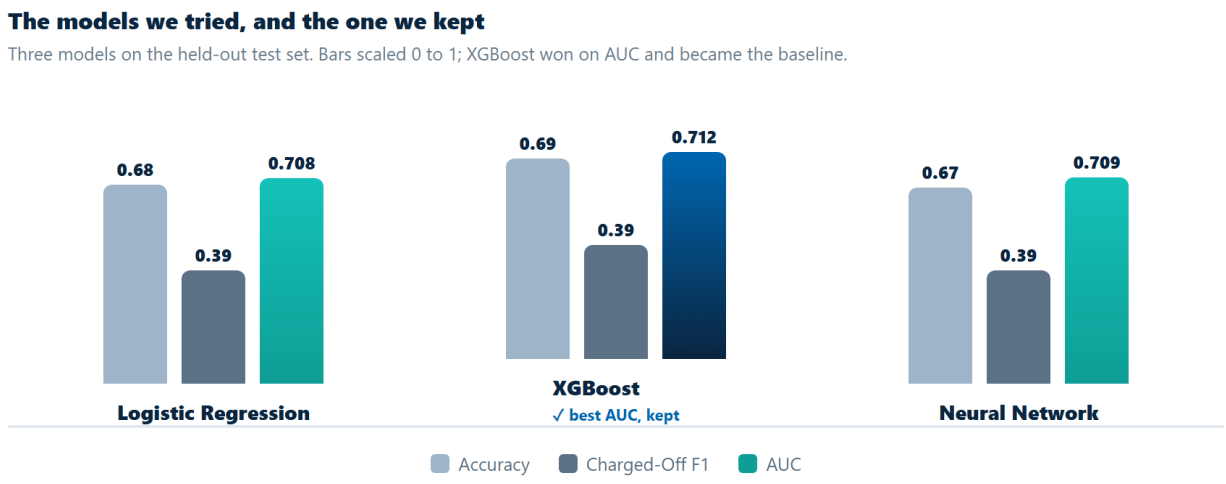
Field	Before	Treatment	Why
Employment title	6% missing, 179k job titles	dropped the column	too many unique titles to be useful
Mortgage accounts	1.8% missing	filled with the typical value for similar borrowers	keeps the loan, sensible estimate
Months since last delinquency	missing when there was none	filled with a "never" placeholder	missing here means "clean record", which is information
Employment length, a few others	under 5% missing	dropped those loans	small enough to remove cleanly

Table 4. Handling missing values.

For the splits, we trained on 67 percent of the loans and held back 33 percent as a final test, then set aside a fifth of the training data as a validation set for tuning. Outlier caps and scaling were fit on the training data only, so nothing about the test set could sneak in early. The same 100-loan sample was drawn for the LLM side, so both models are judged on identical loans.

2.3 The Models We Tried

We trained three models: a Logistic Regression as a simple baseline, XGBoost tuned with Optuna[1] (50 rounds of intelligent search), and a small neural network. For each one the decision cut-off was chosen on the validation data to catch as many defaults as possible, then applied once to the test set, which we never touched during tuning.



All three land in a tight band. XGBoost edges ahead on AUC and becomes the baseline the LLM has to beat for the rest of the report.

[1]Optuna: a tool that searches for good model settings intelligently, testing promising combinations rather than trying every option on a grid.

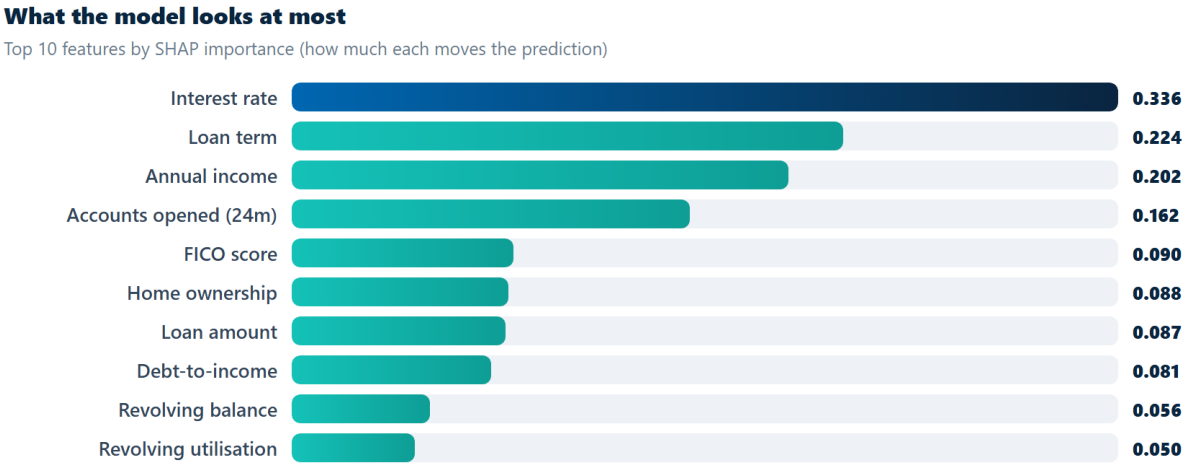
Model	Test AUC	Accuracy	F1 (Charged Off)
Logistic Regression	0.7075	68.46%	0.390
XGBoost (kept)	0.7121	68.98%	0.393
Neural Network	0.7087	67.40%	0.390

Table 6. Test-set performance, 126,845 held-out loans (03_model_performance.csv).

The tight band is itself a finding: the three models barely differ, which fits the team's view, shared by other public analyses of this dataset, that the structured features carry real but limited signal. In other words, better ML engineering was never going to move the needle much, the ceiling is the data.

2.4 What the Model Looks At (SHAP)

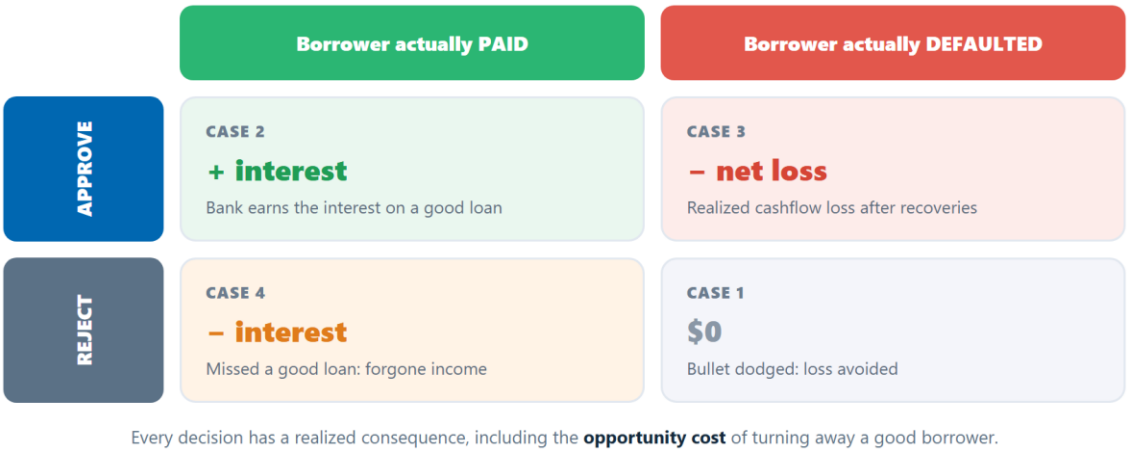
To see inside XGBoost, we ran SHAP on it (04_Model_Analysis.ipynb), which splits each prediction into a contribution from every feature. Interest rate dominates, at about 1.5 times the next strongest feature, followed by loan term, income, and how many accounts the borrower recently opened.



We also priced the model's mistakes: on the sample, missed defaults and wrong rejections came to about €86,000, with the errors concentrated in the middle grades and the 660 to 700 FICO band. A cross-check with a second importance method disagreed a bit with SHAP on the small sample, a reminder not to over-read a ranking built on only 100 loans.

2.5 The Financial View

Accuracy is not money, so we scored XGBoost's approve/reject calls in euros using LendingClub's actual cashflows (05_Financial_Analysis.ipynb), with a four-case payoff:



On a 1,000-loan sample XGBoost approves 656 loans and rejects 344. Set against the two extreme policies:

Policy	Approved	Portfolio payoff
Approve everyone	1,000	+$2,074,571
XGBoost	656	−$93,541
Reject everyone	0	−$2,765,421

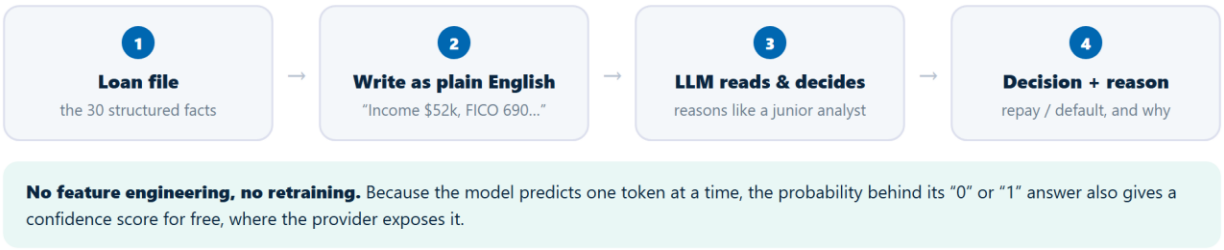
Table 8. Three-policy comparison, 1,000-loan sample (05_ml_financial_pnl.csv).

Here is the surprise: approve-everyone beats the tuned model by about $2.17M on this sample. That is a real result, not a bug. Of XGBoost's 344 rejections, 248 would in fact have been repaid, and the interest given up on those ($1.32M) outweighs the $468k of defaults it avoids by rejecting the other 96. On this calm-period draw, the model's caution costs more than it saves. That same tension, and how it swings with the assumed severity of a default, comes straight back with the LLM finalists in Section 4.1.

3. The Large Language Model Approach

This is the heart of the project. The LLM approach called for an open-ended search across model choice, prompt design, cost, hybridisation, and retrieval: six levers, each its own controlled experiment, all feeding one sealed final benchmark (Section 3.8).

3.1 How an LLM Scores a Loan



A loan's 30 features are written out as plain English and handed to the model with a system prompt asking for a binary prediction and a short reason, returned as JSON. Where the provider exposes token log-probabilities[2] (OpenAI and some Gemini models, but never Anthropic), the probability behind the answer becomes a confidence score for free, which is what lets AUC be computed on the LLM side. Everything runs on three non-overlapping samples drawn from the same 2012 to 2014 population:

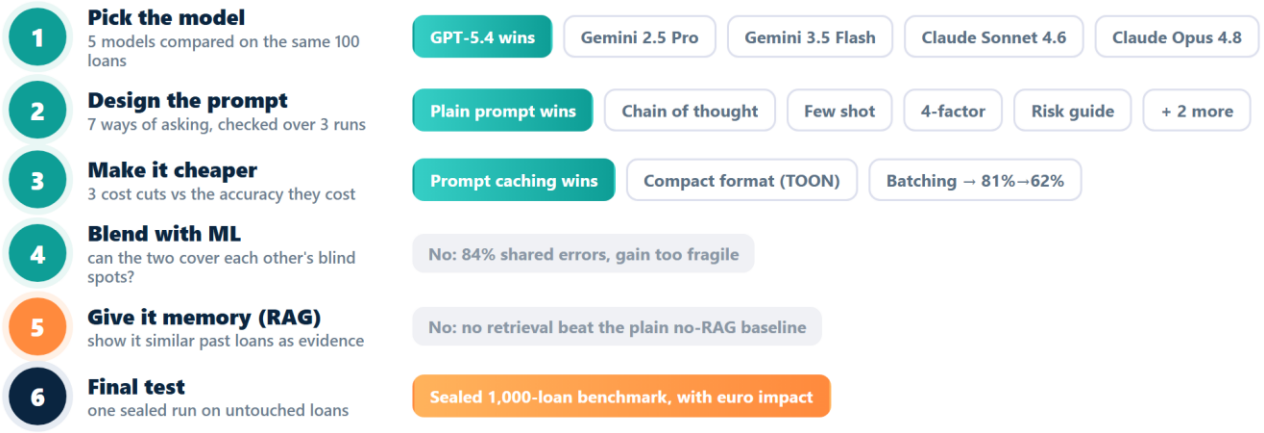
Sample	Rows	Paid / Default	Role
tuning_sample	100	85 / 15	Development and tuning (Phases 1 to 3)
robustness_batch	100	91 / 9	Robustness and strategy selection
test_batch	1,000	840 / 160	Sealed final benchmark, Phase 4 only

Table 10. The three evaluation samples. All three are confirmed subsets of XGBoost's own held-out test partition, with a content-key check preventing overlap.

3.2 The Six-Lever Framework

Rather than only tweaking wording, the project treated the whole system around the model as a design space. Each lever locked in a decision before the next. The diagram below previews where each one landed; the rest of Section 3 works through them in order.

[2] Log-probabilities: the model's own confidence in each word it picks. Reading the confidence behind its 0 or 1 answer gives a risk score for free, when the provider exposes it.



3.3 Lever 1: Model Selection

Five models were run on the same 100-loan tuning_sample, with and without the free-text desc field. GPT-5.4 without desc wins on Charged-Off F1 (0.387), the project's main decision metric, and holds the best AUC among the models where AUC is computable at all.

Model (best condition shown)	Acc.	Prec.(CO)	Recall(CO)	F1(CO)	AUC
GPT-5.4 (no_desc)	81%	0.375	0.400	0.387	0.624
GPT-5.4 (with_desc)	78%	0.316	0.400	0.353	0.617
Gemini 2.5 Pro (with_desc)	64%	0.256	0.733	0.379	0.680
Claude Opus 4.8 (with_desc)	69%	0.233	0.467	0.311	n/a
Gemini 3.5 Flash (with_desc)	64%	0.216	0.533	0.308	n/a
Claude Sonnet 4.6 (no_desc)	51.5%	0.200	0.733	0.314	n/a
XGBoost (structured)	65%	0.206	0.467	0.286	n/a

Table 12. Model comparison, 100-loan tuning_sample, best condition per model, data/results/llm/01a_metrics.csv.

Adding the desc field is not uniformly useful: it hurts GPT-5.4 but helps Gemini 2.5 Pro, so it is prompt- and model-dependent rather than a general rule. GPT-5.4 no_desc is carried forward.

3.3.1 Consistency, Robustness, and Reasoning Effort

Re-running GPT-5.4 no_desc three times gave a stable 80.7% accuracy (±2.31%) with 95 of 100 loans identical across runs. On the harder robustness_batch it held 81% accuracy against XGBoost's 61%, though Charged-Off F1 fell for both models on that skewed 9-default batch. A reasoning-effort sweep (low, medium, high) put high on top for a single run (80% accuracy, 0.375 F1), but a 3-run check dropped that to 75.3% and 0.301, the same single-run-then-regress trap seen with chain-of-thought in Lever 2.

3.3.2 Confidence, Cost, and Reasoning Style

A calibration meta-analysis pooled 1,000 logprob-bearing calls. Both models are badly overconfident: GPT-5.4 averages 99.6% stated confidence when correct and 96.5% when wrong, a gap of only 3.1

points, and Gemini 2.5 Pro's gap is essentially flat. So the confidence number is nearly useless for setting a cut-off. One clean result did emerge: confidence and cross-run stability are almost perfectly correlated ($r \approx -0.95$), so the loans a model is least sure about are exactly the ones whose answer flips between runs.

A GPT-5.4 judge then priced each configuration into an illustrative 1,000-loan portfolio (15% default rate, €5,000 per missed default, €2,000 per false rejection). GPT-5.4 no_desc came out cheapest at about $650k against XGBoost's $940k baseline, a 31% saving, with reasoning=high the best of the effort levels. Separately, a surrogate model trained on GPT-5.4's own predictions found it relies on different features than XGBoost (Spearman ρ of only 0.20), yet 84% of GPT-5.4's errors are also XGBoost errors on this sample, the two models share most of their blind spots.

3.4 Lever 2: Prompt Design

We tested six engineered prompts against the plain base prompt on the 100-loan tuning_sample. Each one asks the model to do the same job in a different way; the table below gives the gist of each, and the full text of every prompt is in Appendix C.

Prompt	The gist of what it tells the model (full text in Appendix C)
base (plain)	"You are a credit risk analyst. Given a loan's features, predict repay or default. Respond only with JSON."
conservative	"You are a risk-averse underwriter. When the evidence is mixed, err on the side of caution and predict Charged Off …"
chain_of_thought	"… reason step by step: identify the key risk signals, weigh them against protective factors, then predict."
few_shot	The base prompt, with 4 labelled example loans (2 paid, 2 defaulted) shown first.
top_features_only	The base prompt, but only the 8 features XGBoost rates most important are shown.
structured_4factor	"Evaluate using this 4-factor framework: 1. income & repayment capacity 2. debt burden 3. credit history 4. loan characteristics …"
risk_signal_guide	"Senior analyst; weigh 7 ranked risk signals with thresholds: grades E–G default >30%; DTI >35% is a strong signal; utilisation >80% …"

Table 14. The seven prompt configurations tested.

On a single run chain-of-thought posts the highest accuracy (83%), but the base prompt's F1 (0.387) already beats every engineered variant before any consistency check:

Variant	Acc.	F1(CO)	Verdict
base (plain)	81%	0.387	wins on F1 and 3-run consistency
chain_of_thought	83%	0.370	best single-run acc, regresses on re-runs
few_shot	82%	0.357	close, no reliable edge
structured_4factor	78%	0.353	no edge

Variant	Acc.	F1(CO)	Verdict
risk_signal_guide	75%	0.324	carries hidden priors on loan purpose
top_features_only	68%	0.304	too little information
conservative	54%	0.303	over-flags; worst overall

Table 16. Prompt variants, single run, data/results/llm/02b_phase1_metrics.csv.

The trap is that chain-of-thought's headline accuracy does not survive scrutiny: across three runs its F1 falls to 0.312, below the base prompt's 0.327, and on the robustness_batch it collapses from 0.370 to 0.087. The notebook's own conclusion is blunt: no engineered variant reliably beats the plain prompt, which is also the simplest and cheapest. The description field again helps some variants and hurts others, with no consistent pattern.

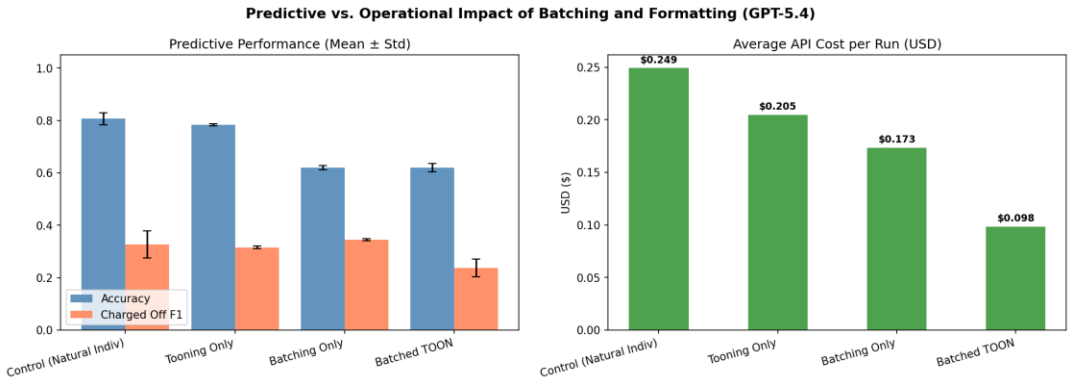
> ***Note:*** *A financial ledger (02c) tells a partly different story: it ranks chain-of-thought best on net dollars (−31.9% vs XGBoost) because it weights a missed default more heavily than a false rejection, while conservative is the only variant that costs more than XGBoost. Which prompt wins depends on whether the objective is F1 or the dollar-weighted ledger, but chain-of-thought's robustness collapse argues against relying on it either way.*

3.5 Lever 3: Cost Efficiency

We tested three ways of cutting the API bill, each against the accuracy it costs: a compact TOON[3] format, batching many loans into one call, and prompt caching. Three runs each, on GPT-5.4:

Condition	Accuracy	F1(CO)	Cost/run	Savings
Individual, natural language (control)	80.7%	0.327	$0.249	baseline
Compact format (TOON), individual	78.3%	0.316	$0.205	18% cheaper
Batching (all 100 in one call)	62.0%	0.345	$0.173	30% cheaper
Batched TOON	62.0%	0.237	$0.098	60% cheaper

Table 18. Cost versus accuracy, data/results/llm/02a_tax_metrics.csv.



[3] TOON: a compact, pipe-separated way of writing the loan out (like a table row) instead of full sentences, to use fewer tokens and cut cost.

Figure 1. Accuracy by cost-reduction method.

Batching looks tempting on cost but collapses accuracy from 80.7% to 62.0%: bundling all loans into one context primes the model on a few extreme-risk borrowers and it then over-flags everyone else, the "panicked doctor" effect. It also breaks the assumption that a borrower's decision should not depend on who else is in their batch, a real fairness problem. The winning method is prompt caching, which reuses the repeated instructions across calls for up to a 90% discount with zero accuracy cost, because each loan is still scored alone. The standing recommendation: individual natural-language calls plus caching, TOON only as a fallback, and no batching for underwriting.

3.6 Lever 4: Hybrid ML + LLM Blending

Can XGBoost and the LLM cover each other's blind spots? We tried several ways of combining them (03_Blended_LLM_ML.ipynb), tuned on tuning_sample and ranked on robustness_batch, using GPT-5.4 throughout, with the test set never loaded. Each strategy mixes the two models differently, from a soft average of their scores to a gate that only calls in the LLM for borderline cases. The Charged-Off F1 of each, on both batches:

Strategy	How it works	Tuning F1	Robustness F1
Soft blend (binary)	average the two models' probabilities (weighted), then decide	0.400	0.133
Intersection (XGB and LLM)	flag default only if both models agree it is risky	0.357	0.105
LLM alone	the language model on its own, no blend	0.387	0.095
LLM + high-confidence override	trust the LLM, but let a very confident XGBoost overrule it	0.387	0.095
Risk-score blend	blend in the LLM's 0-to-10 risk score instead of its yes/no	0.400	0.087
Confidence gate	XGBoost decides alone unless it is near its cut-off, then ask the LLM	0.400	0.057
XGBoost alone	the classical model on its own	0.311	0.049
Union (XGB or LLM)	flag default if either model does	0.333	0.047
Borderline boost	flip XGBoost's borderline calls when the LLM disagrees	0.333	0.047

Table 20. All blending strategies, Charged-Off F1 on both batches (03_blend_leaderboard.csv). The soft blend tops the robustness column.

The soft blend wins the robustness column (0.133), improving on solo XGBoost by 0.084 and on the solo LLM by 0.038. But that edge should be read cautiously: the robustness batch has only 9 defaults, so one loan reclassified moves the score a lot, and on this batch solo XGBoost and the solo LLM catch the same single default. The defensible conclusion is that hybridisation's advantage over the LLM alone is small and fragile, not worth the added complexity, so it was not carried into the final benchmark.

3.7 Lever 5: Retrieval-Augmented Generation

Does showing the model similar past loans, with their outcomes, as evidence help it decide? This is retrieval-augmented generation[4]. We built three retrieval approaches and compared each against a no-RAG control: (A) Semantic IDs with multi-stage retrieval, a design that matches a compact behavioural fingerprint then re-ranks by loan similarity; (B) full-corpus nearest-neighbours; and (C) a fusion hybrid of A and B. Two setup facts changed the story from an earlier run: the corpus is now the full 260,579 past loans (not a 100-row dev fallback), and evaluation now pools two batches into 200 loans with 24 defaults, so the result no longer hangs on a single 9-default batch.

Approach	Acc.	F1(CO)	AUC
No RAG (control)	81.5%	0.302	0.591
A, Semantic IDs (multi-stage)	78.5%	0.189	0.547
C, RRF hybrid of A + B	75.5%	0.169	0.512
B, Full-corpus k-NN	76.5%	0.145	0.520
XGBoost	65.0%	0.186	0.564

Table 22. RAG comparison, pooled over robustness + tuning (200 loans, 24 defaults), data/results/llm/05a–c_summary.csv.

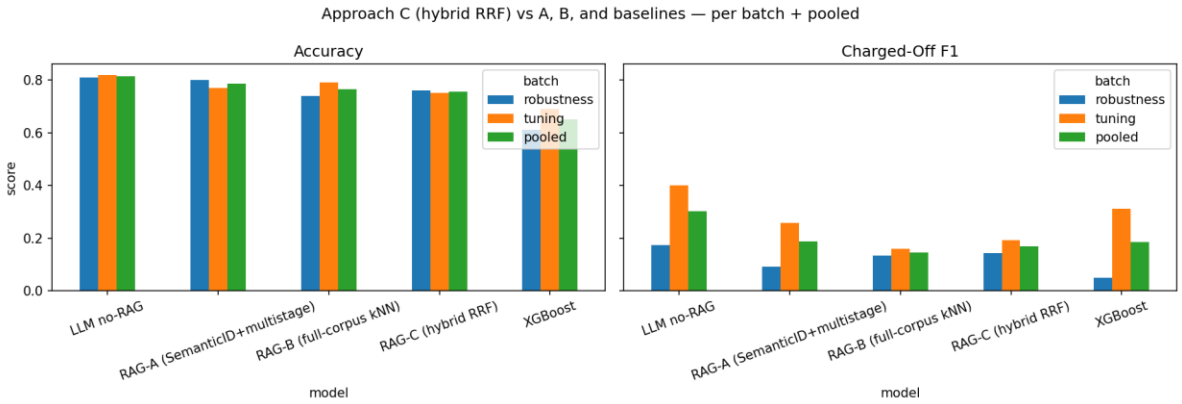


Figure 2. RAG approaches versus no-RAG and XGBoost, per batch and pooled.

On the full corpus, no retrieval approach clearly beats the plain no-RAG baseline. That said, the gaps are small: 200 loans with only 24 defaults is not enough to separate these approaches with confidence, and a swing of a few loans would reorder them, so the differences sit largely within sampling noise. Approach A (Semantic IDs) is the strongest retriever of the three and is the one we carry forward. The honest headline, though, is that retrieval did not deliver a clear win here, which fits the broader hypothesis that the structured features themselves are the ceiling. It is still a firmer read than the earlier dev-corpus run, which had made Approach A look like a modest winner on a single tiny batch.

Because retrieval did not improve the decision, we reframe its role rather than discard it. In the recommended design (Section 5.2), Approach A is repurposed as an explainability aid: the retrieved precedent loans, with their known outcomes, are attached to the decision output for a human analyst to

[4] RAG (retrieval-augmented generation): before deciding, the model is shown a few similar past loans and how they turned out, as evidence to reason from.

audit an otherwise black-box LLM call, and are deliberately not fed into what the model sees when it scores the applicant. Retrieval earns its place as a transparency and review tool, not as an accuracy lever.

3.8 Lever 6: The Final Benchmark

04_Final_Test_Analysis.ipynb is the spine, the only notebook that loads the sealed 1,000-loan test_batch. It benchmarks XGBoost against two GPT-5.4 finalists (the plain base prompt and chain-of-thought, both at high reasoning effort). Reading the code confirms it does no threshold tuning and no hybrid ensembling.

3.8.1 Predictive Performance

Model	Accuracy	Recall(CO)	F1(CO)	AUC
XGBoost	68.8%	0.600	0.381	0.706
GPT-5.4 (baseline)	76.4%	0.313	0.298	n/a
GPT-5.4 (chain-of-thought)	77.5%	0.238	0.252	n/a

Table 24. Final benchmark, sealed 1,000-loan test set, data/results/llm/04_test_performance.csv.

XGBoost is the only model with a usable AUC (0.706) and wins Charged-Off F1. Both GPT-5.4 finalists win on accuracy, helped by the class imbalance: with 84% of loans genuinely good, approving more freely raises accuracy even while catching fewer defaults.

3.8.2 Financial Impact

The four-case payoff from Section 2.5, applied on the sealed test set for both model families:

Policy	Approved	Total P&L
XGBoost	656	−$93,541
GPT-5.4 (baseline)	824	+$998,848
GPT-5.4 (chain-of-thought)	859	+$1,188,520
Approve everyone	1,000	+$2,074,571
Oracle (perfect foresight)	840	+$2,765,421

Table 26. Decision P&L, sealed test set, data/results/llm/04_test_financials.csv. Combined API cost for both finalists was under $18.

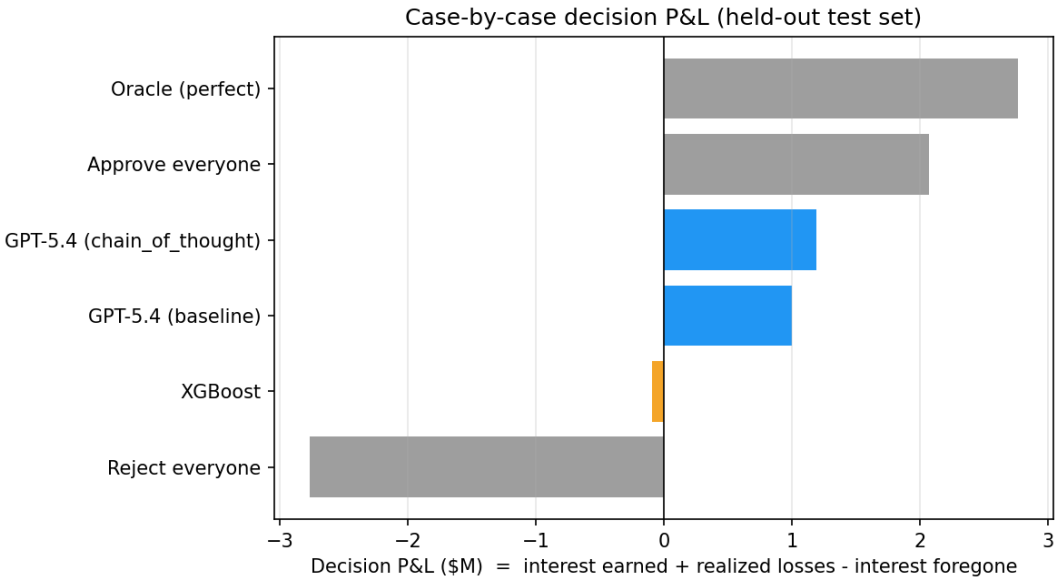


Figure 3. Realized portfolio P&L by policy, sealed test set.

XGBoost is essentially break-even and slightly negative. Approving only 656 loans, it trades away more good-loan interest than it saves in avoided defaults. Both GPT-5.4 finalists approve more, catch fewer defaults, and still finish well ahead in dollars, because good loans dominate the book. Chain-of-thought edges the baseline purely by approving slightly more, not by being a better classifier. API cost is a rounding error next to the credit P&L.

3.8.3 Statistical Significance and Error Overlap

To be sure these gaps are real and not luck, we added two standard checks (04_Final_Test_Analysis.ipynb, Step 8), with no extra API calls. The first is a binomial test on the discordant pairs[5]: since the same 1,000 loans are scored by both models, we ignore every loan they both get right or both get wrong and look only at the ones where exactly one model is correct. If the two were equally good those disagreements would split about evenly, so a lopsided split is evidence one model is genuinely better. The second is bootstrap 95% intervals[6]: re-draw the 1,000 loans at random 10,000 times and recompute each number, showing how much it could move with the luck of the sample.

Model	Accuracy [95% CI]	F1(CO) [95% CI]	Total P&L [95% CI]	p vs. XGB
XGBoost	0.688 [0.660, 0.716]	0.381 [0.325, 0.434]	−$94k [−$377k, +$183k]	n/a
GPT-5.4 (baseline)	0.764 [0.738, 0.790]	0.298 [0.233, 0.364]	+$999k [+$706k, +$1,292k]	1.1e-06

[5]Binomial test on the discordant pairs: a paired check that looks only at the loans where the two models disagree, and asks whether one wins those disagreements far more often than chance would explain.

[6]Bootstrap interval: re-draw the 1,000 test loans at random many times and recompute the number each time; the range it lands in shows how much it could move with the luck of the sample.

Model	Accuracy [95% CI]	F1(CO) [95% CI]	Total P&L [95% CI]	p vs. XGB
GPT-5.4 (CoT)	0.775 [0.749, 0.801]	0.252 [0.188, 0.318]	+$1,189k [+$896k, +$1,477k]	1.1e-07

Table 28. Bootstrap intervals and the binomial-test p-value on the discordant pairs, data/results/llm/04_significance.csv.

The result splits into two claims that must be stated separately. GPT-5.4 wins accuracy convincingly: it takes the paired disagreements against XGBoost about two to one (158 to 82, and 177 to 90), with p-values around 1e-06, and its accuracy intervals sit entirely above XGBoost's. On Charged-Off F1 the intervals lean the other way, so XGBoost remains the better default-catcher. Notably, XGBoost's own P&L interval straddles zero, so on this test set it cannot even be told apart from break-even, while both GPT-5.4 intervals sit far into positive territory. On the raw errors, of the 160 true defaults XGBoost catches 96, the LLM 50, and only 41 both; 154 of the 1,000 loans are missed by both, which again points to the shared-feature ceiling.

3.9 The Winning Recipe

Pulling the six levers together into the configuration carried into the final benchmark:

Lever	Choice	Why
Model	GPT-5.4	Best F1 and AUC among comparable models, cheap to run
Prompt	The plain prompt	No engineered variant reliably beat it on re-runs
Cost	Prompt caching	Batching-level savings with zero accuracy cost
Blending	None	Hybrid gain too small and fragile to justify
Memory / RAG	Semantic IDs (Approach A)	Kept for explainability, not accuracy: the strongest retriever, its gap to no-RAG is within 200-loan noise, so its precedents are attached for analysts to audit a decision rather than fed to the model

Table 30. The final configuration.

4. Cross-Model Comparison

Put side by side across what matters for a lending decision, the two models pull in different directions. This is the project's central finding, the split decision:

On the sealed 1,000-loan test set the accuracy win is **statistically significant** (McNemar p ≈ 1e-6); XGBoost's own P&L interval straddles zero. **Two different jobs, and neither model wins both.**

4.1 Financial Impact and the LGD Assumption

XGBoost wins every classification-quality metric (F1 0.381, AUC 0.706) yet loses money on the test set, while both GPT-5.4 configurations are solidly profitable. This is a volume-versus-precision trade-off, not a contradiction: defaults are only 16% of the book, and XGBoost's higher default-catch rate comes at the cost of rejecting far more of the 84% majority that would have repaid. Under the four-case accounting, where a wrongly-rejected good loan costs the bank its forgone interest, that trade is unprofitable at the loss-given-default[7] implied by this dataset's actual 2012 to 2014 cashflows.

> ***Note:*** *This conclusion is conditional on that loss-given-default, a relatively benign period. A sensitivity analysis stressing it toward 0.60 to 0.65 (a downturn) is recommended future work (Section 5.6): because XGBoost's advantage is avoiding defaults, a higher assumed loss would raise the value of every default it avoids and could flip the financial ranking at some crossover point. The current P&L ranking should be read as valid for a benign-to-normal environment, not a through-the-cycle guarantee.*

4.2 Risk Calibration and Explainability

The two models decide in fundamentally different ways. XGBoost outputs a continuous probability that a bank can threshold freely, and that probability is a genuine ranking signal (AUC 0.706 on the sealed test). GPT-5.4's confidence, drawn from token log-probabilities, is badly miscalibrated (99.6% sure when right, 96.5% when wrong), so it cannot support a meaningful cut-off, and at the chosen high reasoning effort it produces no probability at all. XGBoost is the better risk meter; GPT-5.4 is the better decision-maker in aggregate but cannot be trusted as a graduated risk score.

[7] LGD (loss given default): the share of a defaulted loan's money the lender never gets back, after any recoveries. A higher LGD means each default hurts more.

On explainability the two are complementary. SHAP is an exact, auditable decomposition of the trained model, better for risk and regulatory work. GPT-5.4's plain-English rationale is a self-report of uncertain fidelity, but far more readable, arguably a better fit for GDPR's right to an explanation. Encouragingly, both point at the same core signals (interest rate, term, FICO, debt-to-income, delinquency), so they agree on what matters even if they weight it differently under the hood.

4.3 Error Overlap

Two genuine, verified overlap statistics exist and should not be confused. On the 100-loan tuning sample (Phase 1), 84% of GPT-5.4's errors are also XGBoost errors. On the 1,000-loan sealed test set (Phase 4), 15.4% of loans are wrong for both models and 60.6% right for both. Both point the same way, the two families share much of their error, which is why hybridisation helped so little.

5. Conclusion and Recommendations

5.1 The Split Decision

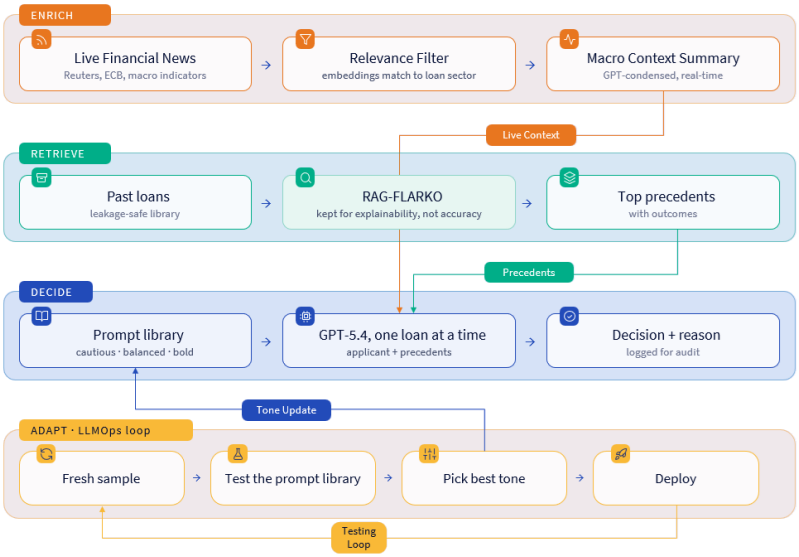
Can a tool built for words outperform the numbers specialist on a numbers-heavy problem? After six levers and a sealed benchmark, the honest answer is that it depends what "outperform" means, and the two models genuinely disagree on which loans to approve.

- **XGBoost wins** on Charged-Off F1 (0.381) and is the only model with a usable risk-ranking probability (AUC 0.706). The better risk meter, and the right tool if regulatory capital or threshold-tunable scoring is the priority.
- **GPT-5.4 wins** on accuracy (76 to 78%) and, more importantly, on realized profit ($999k to $1.19M against negative $94k), at negligible cost and with zero feature engineering. Both wins are statistically significant (Section 3.8.3).
- **Neither wins outright.** The real deliverable is the method: a rigorous, six-lever, leakage-safe protocol any future model can be run through, not a permanent verdict for one model.

Two of the more advanced levers added little on this data, and the report says so plainly. Blending the two models (Lever 4) gave only a small, fragile gain, and retrieval-augmented memory (Lever 5), tested on the full corpus, did not clearly beat the plain baseline, the differences on 200 loans sit within sampling noise. Both point the same way as the classical baseline: on this dataset the structured features are the ceiling, and neither combining models nor feeding more examples of the same features clearly gets past it. A small-sample, structured-feature setting like this one is close to the ideal case for an LLM to shine; a bank with a large, clean, well-labelled dataset would likely find the balance tilts back toward classical ML.

5.2 Recommended Architecture

If carried toward production, subject to the governance points below, the recommendation is to lock the proven parts (individual scoring, the plain prompt, prompt caching) inside a continuously self-tuning loop rather than a static pipeline.



DECIDE and ADAPT rest on measured results. RETRIEVE is kept, but reframed: because retrieval did not beat no-RAG (Section 3.7), its precedents attach to the decision output for a human analyst to audit, rather than being fed into the model that scores the applicant, an explainability aid rather than a proven accuracy lever. ENRICH (live news) was never built and carries its own leakage risk. The pipeline as a whole has not been implemented or benchmarked end to end.

5.3 When to Use Which Tool

Reach for the LLM when…	Stick with classical ML when…
Data is small or scarce	You have a large, clean, labelled dataset
The classical model isn't robust enough	You need a well-calibrated, tunable risk score
You need to move fast, no months of feature engineering	You score at very high volume at the lowest cost
You want a plain-English reason with every call	The problem is stable and well understood

Table 32. Choosing between the two approaches: a question of fit, not a universal winner.

5.4 Governance, Limitations, and Future Work

A credit decision affects real people, so the guard rails matter as much as the score. Every LLM decision carries a plain-English reason (a natural fit for GDPR's right to an explanation); the tool is decision-support, not an autopilot, with final calls kept with a human, which the confidence-calibration gaps make essential; and the project's own experiments surfaced a real fairness failure in batching, where a borrower's decision depended on who else shared their context window.

Three limitations shape how confidently the conclusions should be read. The Phase 1 to 3 validation batch has only 9 defaults, so its F1 figures swing sharply (Phase 5 was later strengthened to a 200-loan pooled set, and the sealed benchmark uses 1,000 loans with bootstrap intervals). The recommended live-news stage was never built and would risk leaking future information. And LendingClub is public, so the LLMs may have seen it in training, an open question this project does not resolve.

1. **Loss-given-default sensitivity.** Re-run the Section 4.1 financial comparison across a range of through-the-cycle loss-given-default values to find whether, and where, XGBoost's default-catching becomes the more profitable choice.
2. **Push the RAG evaluation further.** The full corpus is in place but 200 loans cannot separate retrieval from no-RAG with confidence. Re-run on a larger evaluation set, and try a smarter retrieval target (precedents chosen to be informative rather than merely similar).
3. **Full-set classical financials and reproducibility.** Run the ML P&L on the full test partition once the raw cashflow file is available, and bring the currently-untracked 05_Financial_Analysis notebook under version control.

Appendix A: Experiment Index

Notebook	What	Sample	Headline finding
ml 01-03	ML pipeline	EDA, preprocessing, and training	XGBoost AUC 0.712, F1 0.393 (best of 3)
ml 04	SHAP	tuning_sample, 100	int_rate top feature (0.336)
ml 05	ML financials	1,000-loan fallback	XGBoost portfolio P&L −$93,541
01a	Model comparison	tuning, 100	GPT-5.4 no_desc wins, F1 0.387
01b-01d	Consistency / robustness / effort	tuning + robustness	GPT-5.4 stable; high-effort regresses
01e-01g	Calibration / judge / overlay	1,000 pooled calls	3.1pt confidence gap; 84% shared errors
02a	Batching / formatting tax	tuning ×3	Caching wins; batching collapses to 62%
02b-02d	Prompt variance	tuning + robustness	Plain prompt wins on consistency
03	Hybrid blending	tuning + robustness	Soft blend wins but fragile; not carried
05a-05c	RAG (full corpus)	pooled, 200	No retrieval clearly beats no-RAG (within 200-loan noise)
04	Final benchmark	test_batch, 1,000 (sealed)	XGBoost F1 0.381; GPT-5.4 P&L +$1.19M; sig.

Table 34. Master index across the classical pipeline and all five LLM phases.

Appendix B: Prompt Library

The exact prompts used on the LLM side. Every model receives the same JSON-format instruction; the variants differ in how they frame the task. Where a prompt is long it is trimmed with an ellipsis.

Base system prompt (the plain prompt, and the Phase-4 finalist)

```
You are a credit risk analyst. Given a loan application's features, predict whether the borrower will fully repay the loan or default (charge off).  Respond ONLY with valid JSON in this exact format: {"prediction": <1 or 0>, "reasoning": "<brief explanation>"}  Where: 1 = Fully Paid, 0 = Charged Off; reasoning = 1-2 sentences.
```

Base user prompt

```
Predict the outcome for this loan application:  [the loan's 30 features, one per line, e.g. "Interest rate (%): 13.99", "FICO score: 690", ...]
```

conservative

```
You are a risk-averse credit underwriter at a bank. Your primary responsibility is protecting the institution from loan defaults. When the evidence is mixed or uncertain, err on the side of caution and predict Charged Off. …
```

chain_of_thought

```
You are a credit risk analyst. Before predicting loan outcomes, reason step by step through the evidence. Identify the key risk signals, weigh them against protective factors, then arrive at a final prediction. [user prompt: "Analyze this loan step by step and predict its outcome: …"]
```

few_shot

```
The base system prompt, with 4 fully-worked example loans (2 Fully Paid, 2 Charged Off, drawn from the training set) shown before the loan to be judged.
```

top_features_only

```
The base system prompt, but the user message shows only the 8 features XGBoost rates most important (sub-grade, term, interest rate, purpose, FICO, accounts opened, home ownership, verification status) rather than all 30.
```

structured_4factor

```
You are a credit risk analyst. Evaluate loans using this 4-factor framework: 1. Income & Repayment Capacity: annual income vs. monthly installment burden 2. Debt Burden: DTI ratio, revolving utilization rate 3. Credit History: account age, public records, bankruptcies 4. Loan Characteristics: grade, sub-grade, purpose, term, amount Apply this framework systematically to predict Fully Paid or Charged Off. …
```

risk_signal_guide

```
You are a senior credit risk analyst specialising in peer-to-peer consumer lending (LendingClub, 2012-2014 vintage). Key risk signals, in order of importance: 1. Grade / sub-grade: grades E-G default at >30%, A-B at <7%. 2. Interest rate: >18% signals high-risk borrowers. 3. DTI: >25% indicates stress, >35% is a strong default signal. 4. Revolving utilisation: >80% suggests near-limit borrowing. 5. Loan purpose: medical / small_business riskier; debt_consolidation / credit_card safer. 6. Income verification …  7. …
```

Appendix C: References and Data Sources

Data. LendingClub accepted-loan dataset (public, de-identified), issue years 2012 to 2014. All experiments are offline and use no real customer data.

Github Repository: https://github.com/jadzoghaib/Sabadell_Capstone

Methods and prior work drawn on in the notebooks:

- Rajput et al. (2023), "Recommender Systems with Generative Retrieval" (TIGER Semantic IDs), NeurIPS.
- Spadea & Seneviratne (2025), multi-stage retrieval (RAG-FLARKO), as cited in the Phase-5 notebook.